

实验报告

课程：数字逻辑与计算机组成实验

实验十一：字符交互界面

姓名： 郑袭明

学号： 251502027

日期：2026 年 6 月 11 日

目录

1	设计思路与架构差异	2
1.1	字符显示分辨率与地址计算简化	2
1.2	硬件高效环形滚屏设计	2
1.3	显存读写时序与冲突处理	3
1.4	键盘输入与特殊功能支持	3
2	实验总结	4

1 设计思路与架构差异

本实验实现了一个可以通过键盘输入并在 VGA 显示器上回显的字符交互界面。相比于实验指导书的推荐设计，本实现进行了多项优化，并完成了所有可选的扩展要求（如自动滚屏、Backspace 跨行删除、Shift 键大小写与特殊符号切换、长按键重复输入等）。

1.1 字符显示分辨率与地址计算简化

实验指导书要求采用宽 9 像素、高 16 像素的字模，并在 640×480 的屏幕上显示 70×30 的字符阵列。但在 FPGA 中，对 9 像素宽度的字符进行寻址涉及除以 9 或模 9 的操作，这在硬件上会消耗大量乘除法资源，并引入极长的组合逻辑延迟，严重影响时序收敛。

为此，本实现做出了如下优化：

1. **字符尺寸调整**：将字符宽度改为 8 像素，高度保持为 16 像素。此时屏幕分辨率实际为 80×30 个字符。
2. **字模兼容处理**：直接使用 12 位宽的 `font_rom`，但在显示提取像素时，仅通过 `h_addr[2:0]` 索引低 8 位，自动舍弃了第 9 位（为字符间距的空列）。
3. **无乘除法寻址**：利用 2 的幂次宽度，列地址计算简化为简单的位拼接与移位操作。对于 VGA 控制器输出的当前像素坐标 (h_addr, v_addr) ，其对应的字符 VRAM 单元地址计算为：

$$vram_addr = (vram_row \times 80) + h_addr[9:3] \quad (1)$$

其中 $\times 80$ 可由 $(vram_row \ll 6) + (vram_row \ll 4)$ 的无符号移位加法电路高效实现，规避了乘法器，极大优化了时序性能。

1.2 硬件高效环形滚屏设计

为了支持“输入至最后一行后自动换行，且已输入行自动上移”的滚屏需求，传统的软件做法是整体平移 VRAM 中的数据。但这在硬件上需要一次性拷贝 $30 \times 80 = 2400$ 字节，极其消耗时钟周期与逻辑资源。

本实现采用**硬件级环形缓冲区 (Circular Buffer)** 实现了 $O(1)$ 的零拷贝自动滚屏：

- 引入 1 位寄存器 `offset` 记录是否发生过滚动，以及 `cursor_y` 记录当前光标所在的虚拟 VRAM 行。

- 当光标从最后一行（第 29 行）换行时，不平移显存，而是将 `offset` 置 1，并将光标坐标 `cursor_y` 环形累加。同时，在写入下一行前，将下一行的 VRAM 有效位一次性清空：

$$vram_valid[next_y \times 80 + : 80] \leftarrow 0 \quad (2)$$

- 在 VGA 扫描输出时，根据当前扫描的物理行 `v_addr[9:4]` 与光标位置 `cursor_y` 进行动态地址重映射：

$$y_{sum} = v_addr[9:4] + cursor_y + 1 \quad (3)$$

$$vram_row = \begin{cases} y_{sum} - 30 & y_{sum} \geq 30 \\ y_{sum}[4:0] & y_{sum} < 30 \end{cases} \quad (4)$$

这样，屏幕顶行（物理第 0 行）总是映射显示虚拟的 $(cursor_y + 1) \% 30$ 行，屏幕底行总是映射显示当前光标所在的 `cursor_y` 行，从而在零内存拷贝的开销下实现了平滑的硬件滚屏。

1.3 显存读写时序与冲突处理

字符显存（VRAM）同时被 VGA 扫描逻辑高频读取，又被键盘输入逻辑随机写入。

- **异步/组合读：**由于 VGA 像素时钟为 25MHz，读取显存及字模 ROM 采用了纯组合逻辑路径：

$$vram_addr \rightarrow char_code \rightarrow font_row \rightarrow vga_data$$

此方式能够零延迟响应 VGA 扫描信号，避免了引入同步读带来的时钟延迟偏斜和像素对齐问题。

- **同步写：**键盘模块的数据写入同步于 100MHz 的系统主时钟 `clk`。由于键盘按键的写入频率极低（人类极限输入速度折合几十赫兹），虽然存在读写时钟域的跨越，但实际碰撞概率极小，且即使发生碰撞也仅在极短的一帧内出现某像素的轻微闪烁，不会对视觉造成任何不良影响。这种“异步组合读 + 同步写”的设计在保证显示时序收敛的前提下，极大简化了冲突处理逻辑。

1.4 键盘输入与特殊功能支持

本实现在 `keyboard.v` 和 `cli.v` 中拓展并实现了以下特性：

1. **Shift 键与符号映射**: 通过在键盘状态机中监测左/右 Shift 键的 `make/break` 码, 维护 `shift` 状态信号。在输出 ASCII 时, 利用组合逻辑对小写字母进行大小写转换 (`-8'h20`), 并对数字键和符号键进行重映射 (例如 `8'h31 (1)` 映射为 `8'h21 (!)`), 完整支持了全键盘符号输入。
2. **跨行 Backspace 删除**: 在 `cli.v` 中, 当按下退格键 (`8'h08`) 且光标处于行首 (`cursor_x == 0`) 时, 若当前不是屏幕顶行, 光标将自动回退到上一行的末尾 (`cursor_x <= 79, cursor_y <= prev_y`), 并清除该位置的字符有效位。这实现了跨行删除的连贯交互。
3. **硬件按键重复 (Key Repeat)**: 通过捕获 PS/2 键盘连续发送的相同按键 `make` 码 (不伴随 `break` 码), 在每次接收到新扫描码时累加事件计数器 `key_count`。主控模块 `cli` 仅需检测 `key_count` 的变化即可执行输入, 直接利用了 PS/2 硬件协议的自动重发机制, 免去了在 FPGA 内部设计复杂定时器的繁琐。

2 实验总结

实验时只遇到了一个问题, `scancode_ram.b` 是之前实验提供的, 神奇的使用了同步的写, 导致了一系列问题; 改成异步就都修好了。

主要的时间就花在调试滚屏上了。